Parte 1 · Calentamiento (10 min)

Escribe V o F en cada frase. Se hace solo y sin mirar las claves hasta acabar la parte.

Hora de inicio: _____ Hora de fin: _____

1. start() ejecuta la tarea en el hilo main. __F__
2. Programación concurrente y programación paralela son lo mismo. __F__
3. Dos hilos del mismo proceso comparten memoria. __V__
4. getInputStream() de un Process es lo que el hijo recibe. __F__
5. waitFor() bloquea al padre hasta que el hijo termina. __V__
6. Crear un hilo es más costoso que crear un proceso. __F__
7. Tres hilos de 10, 4 y 5 segundos lanzados con start() tardan 19 segundos en total. __F__
8. Internet es un ejemplo típico de programación distribuida. __V__
9. Por convenio, un código de salida distinto de 0 indica error. __V__
10. Un mismo objeto Thread se puede arrancar dos veces con start(). __F__

Parte 2 · Teoría (30 min)

Responde cada pregunta con tus palabras, en unas 5 líneas y sin apuntes (7 minutos por pregunta).

Hora de inicio: 19:20_____ Hora de fin: 19:41

Pregunta 1

¿Qué diferencias hay entre un proceso y un hilo, y cómo se relacionan?

Un proceso es independiente y dentro tiene hilos, los hilos digamos que son pequeños brazos que se dedican a pequeñas tareas del mismo proceso.

Cada proceso tiene su memoria asignada independiente, y cada hilo comparte la memoria asignada a su proceso

Crear un hilo es más rápido que crear un proceso.

Si un hilo muere por un error puede matar el proceso que lo corre, sin embargo si un proceso muere por algun error es independiente y no afecta a otros procesos

Pregunta 2

¿Qué es la programación concurrente y qué problemas plantea (exclusión mutua y condición de sincronización)? Pon un ejemplo de cada uno.

La programación concurrente es cuando un programa tiene varias tareas (hilos o procesos) funcionando al mismo tiempo, ya sea a la vez en varios núcleos o turnándose en uno solo. El

problema es que muchas veces comparten cosas, como una variable o un fichero, y eso da dos problemas.

La exclusión mutua es que hay recursos que solo puede usar una tarea a la vez, porque si entran dos se estropean los datos. Por ejemplo, si dos hilos sacan 50 € de una cuenta con 100 € a la vez, los dos leen 100 y el saldo se queda en 50 en vez de 0. En Java se arregla con synchronized.

La condición de sincronización es cuando una tarea tiene que esperar a que otra haga algo antes de seguir. Por ejemplo, el padre que hace waitFor() y espera a que el hijo termine, o un consumidor que no puede coger datos hasta que el productor los ha metido.

Pregunta 3

¿Qué diferencias hay entre programación paralela y programación distribuida? Ventajas e inconvenientes de cada una.

La programación paralela es cuando un programa se reparte entre varios núcleos o procesadores del mismo ordenador, que trabajan a la vez y comparten la misma memoria. La programación distribuida es cuando el trabajo se reparte entre varios ordenadores distintos conectados por red, cada uno con su propia memoria, y se comunican mandándose mensajes. Un ejemplo típico es Internet.

La paralela tiene como ventaja que es muy rápida, porque los núcleos se comunican al momento al compartir memoria. Como inconveniente, está limitada por los núcleos que tenga el ordenador, el hardware potente es caro y hay que tener cuidado con la sincronización.

La distribuida tiene como ventaja que se puede ampliar todo lo que quieras añadiendo más ordenadores, y si uno falla los demás siguen funcionando. Como inconveniente, depende de la red, que puede ir lenta o caerse, es más difícil de programar y hay más problemas de seguridad.

Parte 3 · Laboratorio Java (35 min)

Para cada reto: 1) escribe tu predicción, 2) programa, 3) ejecuta y anota qué ha pasado, 4) si no coincide, explica por qué. Si te atascas, usa las pistas por orden. Los programas Eco y Espera están en material/.

Hora de inicio: _____ Hora de fin: _____

Reto 1 · Leer la salida de un proceso (8 min)

Lanza java -version desde Java y muestra lo que imprime, junto con su código de salida.
Predice: ¿lo leerás con getInputStream()?

Mi predicción: Creo que sí, que lo leeré con getInputStream(), porque es la salida del proceso hijo. El código de salida será 0 porque el comando funciona bien.

Qué ha pasado al ejecutarlo: Por getInputStream() no ha salido nada. Todo el texto de la versión openjdk version "25.0.1" etc ha salido por getErrorStream(). El código de salida ha sido 0. (osea que funciona)

Si no coincide, por qué: No coincide porque java -version escribe la versión por la salida de errores y no por la salida normal. Aunque no haya ningún error, ese texto va por getErrorStream(), y el código 0 confirma que el comando ha ido bien.

Ampliación: lanza un comando que no exista y captura la excepción. ¿Qué mensaje da?

Lanza una IOException con el mensaje: Cannot run program "comandoquenoexiste":

CreateProcess error=2, El sistema no puede encontrar el archivo especificado. El proceso ni siquiera llega a crearse, por eso no hay código de salida.

Pistas (solo si te atascas):

- Pista 1: new ProcessBuilder("java", "-version").start().
- Pista 2: mira también getErrorStream(); el código lo da waitFor().

Reto 2 · Enviar datos al hijo (10 min)

El programa Eco lee líneas de su entrada hasta recibir fin y las devuelve en mayúsculas. Haz un padre que le envíe hola, mundo y fin, y muestre lo que responde.
Predice: ¿qué pasa si no envías fin ni cierras el flujo?

Mi predicción: Con fin, Eco devolverá HOLA y MUNDO y terminará con código 0. Si no le mando fin ni cierro el flujo, creo que Eco se quedará esperando más líneas y no terminará nunca, y el padre se quedará colgado en el waitFor().

Qué ha pasado al ejecutarlo: Eco responde HOLA y MUNDO (fin no lo devuelve porque es la palabra para parar) y el código de salida es 0. Al segundo Eco no le mando fin ni cierro el flujo y no termina: a los 3 segundos waitFor(3, TimeUnit.SECONDS) devuelve false y lo destruyo.

Si no coincide, por qué: Sí coincide. Eco se queda bloqueado en readLine() esperando más datos porque el flujo sigue abierto, así que no termina hasta que recibe fin, se cierra el flujo o lo matamos.

Ampliación: espera con waitFor(3, TimeUnit.SECONDS) y, si no termina, destroy().

Con waitFor(3, TimeUnit.SECONDS) el padre no se queda colgado: si a los 3 segundos no ha terminado devuelve false y hago destroy() para matar al hijo. Sale "El segundo Eco no ha terminado en 3 segundos, lo destruyo".

Pistas (solo si te atascas):

- Pista 1: getOutputStream() envuelto en un PrintWriter con autoflush.
- Pista 2: haz close() al terminar. Para lanzar Eco, usa "java", "-cp", System.getProperty("java.class.path"), "Eco".

Reto 3 · Secuencial frente a paralelo (10 min)

Espera duerme los segundos que le indiques. Lanza dos hijos de 2 s: primero uno tras otro, y luego los dos a la vez. Mide el tiempo de cada versión.
Predice: ¿cuánto tardará cada una?

Mi predicción: Uno tras otro tardará unos 4 segundos (2 + 2) porque espero a que acabe el primero antes de lanzar el segundo. Los dos a la vez tardarán unos 2 segundos porque duermen al mismo tiempo.

Qué ha pasado al ejecutarlo: Secuencial: 4265 ms. Paralelo con 2 hijos: 2127 ms. Lo que sobra de los 4 y 2 segundos es lo que tarda en arrancar la máquina virtual de Java de cada hijo.

Si no coincide, por qué: Sí coincide. En paralelo hago start() de los dos antes del primer waitFor(), así que los dos hijos duermen a la vez y el tiempo total es el del más lento.

Ampliación: ¿cuánto tardan 3 hijos de 2 s en paralelo?

3 hijos de 2 s en paralelo tardan 2135 ms, casi lo mismo que 2, porque todos duermen a la vez.

Tarda lo que el más lento más el arranque de cada java.

Pistas (solo si te atascas):

- Pista 1: System.currentTimeMillis() antes y después.
- Pista 2: en paralelo, haz start() de los dos antes del primer waitFor().

Reto 4 · Hilos con Runnable y join() (7 min)

Crea 3 hilos que duerman 1, 2 y 3 segundos e impriman su nombre al acabar. El main debe imprimir cuánto ha tardado en total cuando terminen los tres.
Predice: ¿cuánto tarda? ¿Y si pones el join() justo después de cada start()?

Mi predicción: Con los join() al final tardará unos 3 segundos, lo que tarda el hilo más lento, porque los tres duermen a la vez. Con el join() justo después de cada start() tardará 6 segundos (1 + 2 + 3) porque el main espera a cada hilo antes de lanzar el siguiente.

Qué ha pasado al ejecutarlo: Con los join() en un segundo bucle: 3014 ms, y los hilos acaban en orden Hilo-1, Hilo-2 e Hilo-3. Con el join() después de cada start(): 6025 ms.

Si no coincide, por qué: Sí coincide. join() bloquea al main hasta que ese hilo termina; si lo pongo justo después del start() los hilos ya no van a la vez y se ejecutan uno detrás de otro.

Ampliación: quita los join(). ¿Qué imprime ahora el main y cuándo?

Sin join() el main imprime "Sin join, el main acaba en: 1 ms" nada más lanzarlos, antes de que termine ningún hilo, y luego van saliendo los mensajes de los hilos al segundo 1, 2 y 3. El programa no acaba hasta que terminan los hilos aunque el main ya haya terminado.

Pistas (solo si te atascas):

- Pista 1: una clase que implemente Runnable y reciba los segundos.
- Pista 2: guarda los hilos en un array y haz los join() en un segundo bucle.

Parte 4 · Simulacro (75 min)

Respeta el tiempo: cuando se acabe, para y guarda lo que tengas.

Teoría (25 min, sin internet): ua1ex1_teoria.txt

Hora de inicio: _____ Hora de fin: _____

Respuestas: Las respuestas completas están en ua1ex1_teoria.txt.

a) Programación concurrente: varias tareas (procesos o hilos) en marcha a la vez cuyas instrucciones se intercalan, en varios núcleos o turnándose en uno, colaborando o compitiendo por los recursos.

b) Hilo frente a proceso: el proceso tiene su memoria y es independiente; el hilo es parte de un proceso (todo proceso tiene al menos uno), comparte su memoria, se crea más rápido y si falla puede tirar el proceso.

c) Exclusión mutua: un recurso que solo puede usar uno a la vez (sección crítica), por ejemplo dos hilos sacando dinero de la misma cuenta. Condición de sincronización: una tarea espera a otra, por ejemplo el padre con waitFor() o productor y consumidor.

d) Paralela: varios núcleos del mismo ordenador con memoria compartida, rápida pero más difícil y limitada al hardware. Distribuida: varios ordenadores por red con mensajes, escalable y con alta disponibilidad, pero depende de la red y tiene más problemas de seguridad.

Práctica (50 min): ua1ex1_practica.txt

Crea Hijo.java y Padre.java.

Hora de inicio: _____ Hora de fin: _____

Hijo.java comprueba el argumento (sin argumento sale con 1, si no es un número con 2 y si es negativo con 3) y si está bien escribe de 1 a n esperando 200 ms entre cada número y sale con 0. Padre.java arranca los 4 hijos con "4", "6", "-2" y "abc" en un bucle y después, en otro bucle, lee la salida de cada uno con getInputStream(), suma los números y espera su código con waitFor(). Captura IOException, InterruptedException y NumberFormatException.

Resultado al ejecutarlo: Hijo 1 (arg=4): codigo 0 -> OK, suma=10. Hijo 2 (arg=6): codigo 0 -> OK, suma=21. Hijo 3 (arg=-2): codigo 3 -> numero negativo. Hijo 4 (arg=abc): codigo 2 -> no es un numero. Suma total: 31.

Todo el programa tarda unos 1,4 s: como los hijos van a la vez, tarda lo que el de 6 números (6 x 200 ms) más el arranque de java, no la suma de todos.

Parte 5 · Autoevaluación

Con las claves de claves/ rellena esta hoja.

V/F (10 frases)	10 /10	Qué fallé: Nada.
Teoría: proceso frente a hilo	2 /3	Qué fallé: Me faltó decir que un proceso tiene como mínimo un hilo y que los hilos se comunican por la memoria compartida sin pasar por el sistema operativo.
Teoría: concurrencia y problemas	3 /3	Qué fallé: Nada.
Teoría: paralela frente a distribuida	3 /3	Qué fallé: Nada.
Reto 1	OK	Qué fallé: Pensaba que java -version salía por getInputStream() y sale por getErrorStream().
Reto 2	OK	Qué fallé: Nada.
Reto 3	OK	Qué fallé: Nada.
Reto 4	OK	Qué fallé: Nada.
Simulacro teoría	9 /10	Qué fallé: En la a) podría haber puesto un ejemplo de programa concurrente.
Simulacro práctica	10 /10	Qué fallé: Nada, la salida coincide con el ejemplo del enunciado.

Lo que voy a repasar antes del examen:

1. fork(), wait() y getpid()/getppid() en C, sobre todo un padre con varios hijos (el examen práctico es en C).

2. Señales (signal, kill, pause) y tuberías (pipe, read, write) para sincronizar y comunicar procesos en C.

3. Las diferencias proceso/hilo y paralela/distribuida con ventajas e inconvenientes, para la teoría.